

Introduzindo os métodos formais

Os métodos formais visam complementar os processos de teste e revisões utilizando matemática a fim de analisar o comportamento de um software. Nesse caso, ao invés de validarmos diferentes testes de tipos de cenários, eles comprovam matematicamente que um sistema se comporta conforme esperado.

Os processos são descritos inicialmente em linguagem formal com uma descrição matemática de como deveria ocorrer o comportamento para os cenários sem efetivamente implementá-lo com linguagem natural, algumas linguagens populares para isso são: **Z Notation**, **VDM (Vienna Development Method)**, e **B-Method**. Depois de estruturar o fazemos a verificação Formal e do Modelo, técnicas para comprovar a tese matemática e garantias de persistência de propriedades do método respectivamente.

Alguns exemplos de usos para os métodos formais são

1. **Programas Espaciais da NASA:** Métodos formais têm sido usados no desenvolvimento de software para as missões espaciais da NASA para garantir a confiabilidade de componentes críticos de segurança.
2. **Sistemas de sinalização ferroviária:** Em sistemas ferroviários europeus, os métodos formais garantem que o software de sinalização do trem funcione com segurança em todas as circunstâncias, evitando acidentes.
3. **Contratos inteligentes em Blockchain:** Métodos formais são cada vez mais usados para verificar a correção de contratos inteligentes em plataformas blockchain, reduzindo o risco de vulnerabilidades e bugs.

3 ferramentas para Modelos Formais em ES

1) TLA+

Linguagem formal para **especificar e verificar sistemas concorrentes e distribuídos**.

Baseada em lógica temporal, permite modelar o comportamento do sistema ao longo do tempo e encontrar erros antes da implementação.

Pontos-chave:

- Ideal para sistemas distribuídos complexos.
- Detecta bugs de concorrência e inconsistências lógicas.
- Muito usada por empresas como a AWS para validar sistemas críticos.

2) Alloy

Linguagem declarativa baseada em lógica de primeira ordem para **modelar e analisar estruturas e restrições de sistemas**.

Pontos-chave:

- Foco em modelagem estrutural.
- Permite análise automática via Alloy Analyzer.
- Trabalha com verificação de escopo finito, mesmo em modelos conceitualmente infinitos.
- Muito usada para explorar e validar modelos conceituais rapidamente.

3) Coq

Assistente de provas formais para **definir matematicamente algoritmos e provar propriedades com verificação por máquina**.

Pontos-chave:

- Permite desenvolver provas matemáticas formais.
- Usado para certificação de software crítico.
- Aplicado em projetos como o CompCert (compilador C formalmente verificado).

Um caminho para o Futuro da ES

Copilot ajuda a implementar, mas não garante qualidade

Ferramentas como o GitHub Copilot facilitam a escrita de código (implementação), mas **não resolvem especificação nem verificação**. Isso pode levar a bugs sutis se o código for aceito sem validação adequada.

Engenharia de software vai além de escrever código

O processo envolve:

- **Especificar** o que o sistema deve fazer
- **Implementar**
- **Verificar** se está correto
- **Iterar**

O texto defende que devemos fortalecer principalmente **especificação e verificação**, pois são elas que garantem qualidade, segurança e confiabilidade.

O futuro está nos métodos formais

Métodos formais ajudam a especificar e verificar sistemas com mais rigor. Exemplos citados:

- TLA+ (verificação de sistemas distribuídos)
- Testes baseados em propriedades (ex: Hypothesis)
- Tipagem estática e refinement types

A ideia central: **quanto melhores forem especificação e verificação, mais rápido e seguro será o desenvolvimento.**

O que espero dessa disciplina

Acredito que a disciplina será um grande diferencial para um futuro em que iremos implementar código cada vez mais rápido e, conseqüentemente, teremos que especificar, verificar e validar funcionalidades estruturais.